专利技术交底书

发明名称：一种结合历史访问与当前批次阶段信息的混合专家模型联合显存分配方法

技术联系人：_____

联系人电话：_____

联系人邮箱：_____

1、你的 idea 要解决的是什么技术问题？

本发明涉及大模型推理系统的内存管理，适用于专家权重与键值缓存不能同时全部常驻 GPU 的混合专家模型推理场景。专家权重用于执行被路由选中的前馈计算，键值缓存用于复用已经处理的上下文；两者共同竞争扣除固定模型部分、激活和临时空间后剩余的显存。给专家池增加容量可能减少专家 miss，却压缩 KV 池并增加 KV 恢复需求；反向调整也具有相同的资源竞争关系。

拟解决的技术问题是：在请求组成动态变化、同类 miss 存在多种处理路径的情况下，如何以较低的在线决策开销，为专家权重和 KV 缓存确定可执行的显存划分，减少由缓存缺失带来的恢复开销。具体包括历史访问统计对当前批次变化响应滞后，以及统一的单次 miss 成本难以区分不同处理路径这两个问题。

本发明采用历史访问和当前批次信息联合决策。历史部分分别维护专家和 KV 的复用距离统计以及四条路径的计时记录，给出考虑恢复代价的分配建议；当前部分读取已经排程但尚未执行的 Prefill 与 Decode 构成，生成阶段修正建议；随后将两种建议融合并转换为满足显存约束的容量。执行结果再反馈到历史记录。阶段信息用于修正资源需求，不通过 GPU 执行时间重新定义工作负载阶段。

2、解决该技术问题最接近的现有技术是怎么做的（背景介绍）？ 最接近的现有技术有什么技术缺点？

2.1 相关技术及工作方式

常见的卸载式推理系统将部分专家权重或 KV 缓存保存于 CPU 主存，在 GPU 需要对象时进行搬运或重新计算。固定显存划分简单，但在上下文增长或请求组成变化时，可能使一种资源具有冗余容量而另一种资源频繁发生 miss。仅根据当前 miss 数调整容量也无法直接判断其他候选容量的效果，因为当前 miss 是在当前容量和状态下形成的观测。

WiSP 从工作集角度研究显存受限的 MoE 推理，将专家缓存与 KV 池放在统一可分页预算下，并根据两类资源的容量收益进行联合分配。其重点场景是低并发服务，但论文也涉及启动阶段的 Prefill/Decode 分析与在线控制，不能将其概括为完全未考虑 Prefill。本发明与该类技术的主要区别在于：在历史容量收益分析之外，显式引入下一批次已知的 P/D 构成进行分配修正，并将专家与 KV 的不同恢复路径纳入单次 miss 代价模型。

Fiddler 研究 CPU–GPU 协同执行 MoE 推理，说明专家 miss 不必总是将完整专家权重传入 GPU；也可以在 CPU 上执行专家计算后，仅回传输出。该技术为专家多路径处理提供已有技术基础，但 CPU–GPU 路径选择本身不等于专家池与 KV 池的联合显存分配，也不等于历史分配和当前批次阶段信号的融合。

2.2 与本发明相关的技术不足

第一，仅依赖历史窗口中的访问特征时，若下一批次的请求组成已经变化，分配结果仍可能偏向上一窗口。若改用观测到的 Prefill/Decode GPU 时间划分阶段，则时间又受容量和执行路径影响，阶段判断可能随自身控制动作改变，不能独立反映排程输入。

第二，以每类对象的一个固定 miss 成本作为代理值，不能完整反映路径异构性。固定大小权重上传或 KV 恢复主要受传输影响，而专家 CPU 路径受路由 token 数影响，KV 重计算受恢复范围和上下文影响。在不同运行状态下，较低代价的路径可能不同，单纯减少总 miss 数也未必减少总恢复时间。

第三，将 Prefill 与 Decode 的历史统计完全割裂，容易忽略两者共用物理缓存产生的跨阶段复用。与此同时，复用距离直方图对应的是规定缓存语义下的容量收益；当 CPU 计算等路径绕过缓存准入时，不能无条件把参考 LRU 曲线当成实际缓存 miss 的精确结果。本发明保留统一访问历史，并明确区分预测参考与实际执行状态。

3、解决该技术问题（你的产品）你的 idea 是怎么做的？（这部分最重要，要详细写明 How to do，而不仅仅停留在一个 idea，最好有图，用文字解释图）

3.1 系统组成及总体流程

本方法运行于具有 CPU 主存和 GPU 显存的推理服务器，由推理运行时提供请求阶段、对象访问、路由结果和缓存状态。系统包含历史观测模块 101、批次信息模块 102、历史分配模块 103、阶段修正模块 104、融合与执行模块 105，以及在线更新模块 106。各模块可以作为同一推理服务进程中的软件组件实现，不要求新增专用硬件。

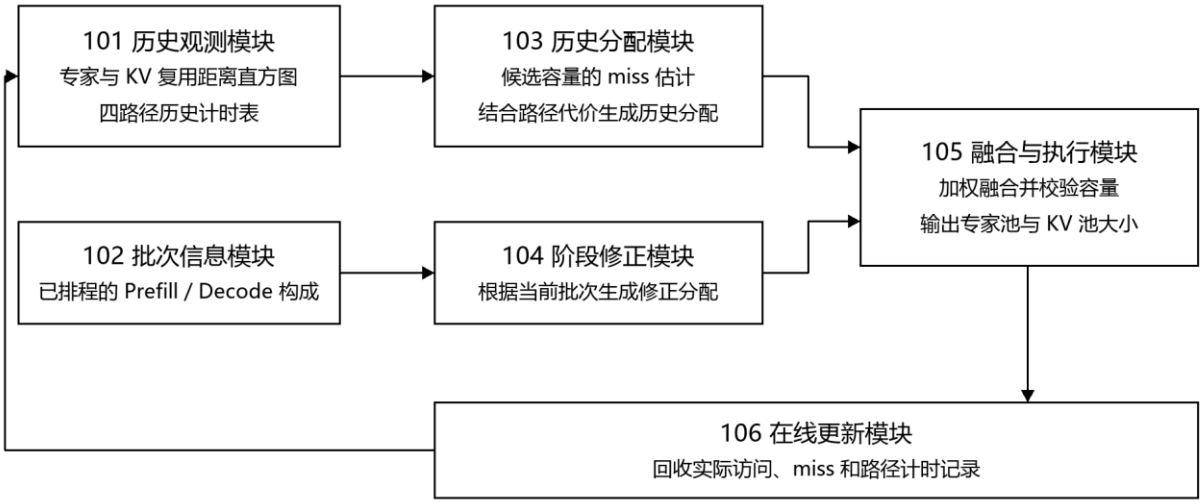


图 1 历史与当前批次联合驱动的显存分配系统

如图 1 所示，历史观测模块向历史分配模块提供两类资源的容量收益估计及恢复路径代价；批次信息模块向阶段修正模块提供当前即将执行批次的 P/D 构成。两种分配建议汇入融合与执行模块，该模块输出专家池和 KV 池的容量，而不是重新生成请求或修改用户输入。在线更新模块在实际执行后回收观测结果，供下一轮使用。

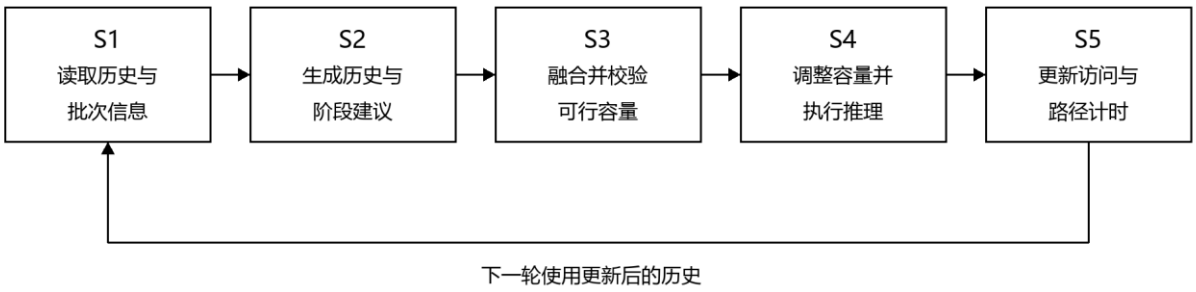


图 2 联合显存分配及在线更新流程

图 2 中的 S1 至 S5 分别表示读取历史和批次信息、生成历史与阶段两种建议、融合并投影到可行容量、安全调整并执行推理、更新历史统计。一个决策周期可以是一轮排程，也可以由若干轮组成；同一实现中保持周期和统计窗口定义一致。

3.2 管理对象 容量及记录字段

专家对象由层标识和专家标识确定，包含该专家前馈计算所需的权重矩阵。KV 对象由请求标识、层或层组标识、token 区间和缓存版本确定，包含对应区间的 K/V 张量。

对象内容、精度和位置编码信息保持一致，不能将不同请求或不同区间的数据混用。每类资源在一种配置下采用固定对象粒度；两类对象不要求具有相同字节大小。

以 B 表示扣除固定非专家权重、激活、临时工作区和安全预留后的联合可分页显存预算，以 B_E 和 B_K 分别表示专家池与 KV 池容量，满足 B_E 与 B_K 之和不超过 B。以 s_E、s_K 表示两类对象的字节大小，容量对应的对象数分别为 B_E 除以 s_E、B_K 除以 s_K 后向下取整。对齐或取整产生的余量不视为已经缓存了对象。

候选划分同时满足当前不可驱逐对象、传输缓冲区和临时专家执行槽等最低容量约束。对于被正在执行的 kernel 或异步传输引用的对象，设置锁定标记，不能在调整容量时直接释放。若可用显存不足以满足最低约束，维持安全配置或等待资源释放，不强行套用分配结果。

记录类别	主要字段	作用
访问记录	周期编号、资源类型、对象标识、访问顺序、对象字节数	维护统一资源访问历史及复用距离
批次记录	批次编号、Prefill 请求数、Decode 请求数、实际 query token 数	在执行前提供阶段修正信号
路径记录	对象类型、处理路径、路由 token 数或上下文长度、完成时间、精度	更新单次 miss 代价模型
缓存记录	实际容量、驻留对象、锁定状态、准入及驱逐结果	保证实际状态与执行正确性

3.3 统一历史及候选容量的 miss 预测

分别为专家对象与 KV 对象维护访问顺序和复用距离直方图。Prefill 与 Decode 对同一类资源的访问进入同一条序列，不建立两个互不共享的物理缓存模型。访问指推理执行器对管理对象的逻辑需求；专家侧对应实际非空专家调用，KV 侧对应缓存管理器定义的块读取需求，不能仅依据上下文长度虚构已观测到的物理地址访问。KV 新建写入用于更新驻留状态和参考顺序，不计为需要 Restore/Recompute 的 miss；只有需要复用而未驻留的读取才计入恢复 miss 曲线。

一个对象再次被访问时，统计从其上次访问之后到本次访问之前出现过的不同对象数，记为复用距离 d；首次访问单独记录为冷访问。以 C 表示某类缓存可容纳的对象数，在等大小对象、全相联 LRU 且每次 miss 均准入等条件下，冷访问以及 d 不小于 C 的访问发生 miss。例如序列 A、B、C、A 中，最后一次 A 的复用距离为 2；容量为 2 时该次访问 miss，容量为 3 时该次访问 hit。

统计窗口内，将冷访问数与复用距离不小于候选容量的直方图计数相加，即得该段序列在参考 LRU 语义下的 miss 数。对全部候选容量重复累加即可形成 miss 曲线，不必为每个容量实际创建一个 GPU 缓存。为保留窗口边界前的缓存历史，跨窗口继续维护访问顺序；仅统计计数使用滑动窗口，不能每轮把全部对象重新标为首次访问。

预测未来时，以最近窗口的参考 miss 率或单位访问 miss 数结合历史平均访问量，得到下一决策周期的专家 miss 估计 M_E 和 KV miss 估计 M_K。该外推依赖时间局部性，不读取下一批实际对象标识、实际路由结果或实际 miss。冷启动样本不足时，使用预设安全划分并逐步积累历史。当前批次差异由后续阶段修正处理，而不是在直方图中额外建立 P/D 历史桶。

复用距离曲线是候选容量的参考估计，不等于在所有 fallback 路径下都精确预测物理缓存。采用 CPU 计算后不将专家权重准入 GPU 时，或者恢复对象的附带准入规则不符合 LRU 时，实际状态可能偏离参考状态。因此同时保存实际缓存状态与实际 miss，在线检查预测偏差；偏差持续超过预设阈值时，降低历史建议权重或暂时保留当前安全划分。该处理不改变统一历史与批次修正的主线，但不宣称解决了任意缓存策略下的精确反事实预测。

3.4 四条 fallback 路径及单次 miss 代价模型

miss 次数模型和 cost 模型分别维护。路径模型以单个固定粒度对象为单位，使用历史完成时间标定。阶段名称不是路径 cost 模型的直接输入；专家 CPU 路径主要使用路由到该专家的 token 数 q，KV 重计算主要使用按恢复边界定义的上下文长度 L。对象大小、数值精度、CPU 线程数、传输缓冲方式和重计算前置状态作为配置固定或分别标定。

对象及路径	处理过程	低开销模型及边界
专家 Upload	主存权重传入 GPU，在 GPU 执行专家计算	固定大小权重的纯上传近似常数；完整恢复还包含 GPU 计算
专家 CPU	输入传到 CPU，执行专家计算，输出回传 GPU	在标定范围内按 q 拟合带固定项的线性模型，包含输入输出搬运
KV Restore	将主存中的同一 KV 对象传回 GPU	固定大小对象的纯恢复近似常数；要求存在有效主存副本
KV Recompute	从有效前置状态重新生成所需 KV 对象	在固定恢复协议及标定范围内按 L 拟合带固定项的线性模型

专家两路径必须从相同任务起点计时，以同一结果在 GPU 可用为终点。完整 Upload 路径包括权重搬运和 GPU 计算，完整 CPU 路径包括输入 D2H、CPU 计算、必要的格式转换和结果 H2D。若分配目标使用额外 miss penalty，则从两条完整路径中统一扣除同任务的常驻专家 GPU 计算基线。此时 Upload 的额外项可近似视为固定上传时间，CPU 路径的额外项在标定范围内拟合为固定项加 q 的比例项，不能将纯上传时间与完整 CPU 路径未经处理直接比较。

KV 两路径恢复同一请求、同一 token 区间、同一层组的 K/V。Restore 的前提是主存副本有效。Recompute 的前提是保存了原 token、位置和恢复所需的前置缓存或检查点，并能按原模型计算出目标对象；缺少前置缓存时须计入实际需要重算的前缀，不能把一层投影时间当作完整恢复时间。恢复协议变化时使用另一组标定参数，不能仅改变 L 而沿用不同协议的系数。

常数和线性模型都是有适用范围的近似。带宽争用、CPU 并行度、矩阵规模、上下文增长和算子切换可能改变曲线，拟合时保留固定开销并记录残差。路径记录表可维护每条路径的样本数、常数或斜率与截距、适用范围和最近更新周期。模型预测超出标定范围或误差过大时，采用保守历史值或分段查表，不预设任意 q 或 L 下都严格线性。

对每类对象，在满足执行前提的路径中估计单次恢复开销较低的路径。历史分配时，尚未产生的 q 采用过去观测的代表值或分布估计；路径实际执行时才允许使用 router 已经产生的真实 q。对于 L，可以使用执行前已经可知的请求上下文信息。多个 q、L 的样

本分别求路径开销后再汇总，避免把平均参数上的最小值误当成逐对象选择后的平均开销。

计费不重复计算同一工作。KV 宏重计算中若触发内部专家上传，相关时间包含在该宏路径 cost 内，并以恢复事件标识关联内部操作，不再作为同一决策周期的独立主任务专家恢复项重复相加。一次重算产生多个目标块时，共享的前缀工作只计一次；若逐块独立可加假设不成立，按固定恢复对象组标定，或者将共享恢复作为替代实施方式处理。

3.5 生成基于历史和路径代价的分配建议

从可行专家池容量中生成候选集合，余下预算供 KV 池使用，并按各自对象大小取整。对每个候选划分，从两条 miss 曲线取得 M_E、M_K，结合历史路径记录得到两类对象的平均有效单次 miss 开销。以专家预计 miss 数乘以专家有效开销，加上 KV 预计 miss 数乘以 KV 有效开销，形成该候选划分的估计恢复成本；内部共享恢复按上一节口径扣除重复项。

选取估计恢复成本较低的候选划分作为历史建议。以 f_h 表示历史建议中分给专家池的预算比例，KV 比例为其余部分。若多个候选的差异在预设容差内，可优先保留当前划分以减少迁移。该步骤是在给定历史和简化成本模型上的候选比较，不代表对未知未来请求求得理论全局最优。

3.6 根据当前已排程批次生成阶段修正建议

在本轮计算开始前，从调度器读取已排程批次中 Prefill 请求数 N_P 与 Decode 请求数 N_D。Prefill 指处理新输入或一个已排程的输入分块，Decode 指使用已有上下文生成后续 token 的计算；阶段来自请求状态，而不是通过该轮 GPU 时间占比推断。将 N_P 占两者总数的比例作为当前信号 p，Decode 的比例为剩余部分。空批次不产生新分配。

预先确定 Prefill 倾向的专家比例 f_P 和 Decode 倾向的专家比例 f_D，其中 f_P 大于 f_D，二者均落在可行范围内。按 p 在两个比例之间插值，得到当前批次建议 f_c。直观上，Prefill 请求比例较高时，建议给专家更大容量；Decode 请求比例较高时，建议给 KV 更大容量。这是一种低开销的资源需求修正先验，不断言每个 Prefill 一定更需要专家容量，或者每个 Decode 一定更需要 KV 容量。

f_P、f_D 及信号口径由配置或独立标定确定，评价阶段冻结。采用请求数的实施方式成本低，但一个长 Prefill 与一个短 Prefill 具有相同权重，可能造成误差；替代实施方式可

以使用批次内已知 query token 数或其归一化权重。本实施方式不以 GPU 执行时间计算比例，因此不会因分配动作缩短某阶段时间而直接改变该阶段的排程标签。

3.7 融合两种建议并执行安全显存调整

以 λ 表示当前批次建议的权重，取值在 0 到 1 之间。将历史专家比例 f_h 按权重 1−λ、当前专家比例 f_c 按权重 λ 求加权平均，得到融合比例 f。λ 为配置参数，样本充足时按冻结参数运行；冷启动或参考曲线误差过大时可使用事先规定的安全权重。λ 等于 0 表示只使用历史建议，等于 1 表示只使用当前批次建议。

将 f 乘以预算 B 得到专家池的目标字节容量，再将剩余容量分给 KV 池。经过容量边界裁剪、块大小取整和锁定对象检查，形成可执行划分。预算不足、对象对齐或实际锁定状态可能使最终比例不完全等于 f，系统记录目标容量和实际容量，不能只记录加权比例。

在安全同步点应用新容量。缩小某池时，仅驱逐允许移出的对象：专家权重保留可访问的主存来源；KV 若后续还会用到，则保存有效主存副本或保留合法重计算前提。扩容只增加可用槽位，之后由实际访问逐渐填充，不假定扩容立即形成已预热的缓存。若对象仍被 kernel 或传输引用，则延后调整，不释放在用数据。

容量变化很小时可保持现有划分；需要调整时，可以限制每次改变量和最短调整间隔，避免分配器因微小噪声频繁搬运。调整引发的迁移、同步及预热开销单独记录，与 fallback 恢复开销保持一致的评价口径。随后执行已排程批次，发生实际 miss 时根据可用路径、真实 q 和有效恢复状态执行对应路径。

3.8 执行反馈与在线维护

本轮完成后，在线更新模块维护统一资源访问序列、直方图计数和实际 miss 记录，并将每个恢复事件的路径、完成时间、q 或 L 写入路径历史表。传输与计算存在异步提交时，在结果真正可用的完成点结束计时，不能仅用提交函数的返回时间更新 cost。标定和评价中保持精度、缓冲区分配是否计入、CPU 线程数及同步边界一致。

更新只使用已经发生的事件。下一轮先读取冻结的历史版本，再生成建议，避免一个决策周期内边计算边修改其评价依据。主任务访问与 fallback 内部访问使用事件类型区分；实际缓存状态按真实准入和驱逐结果更新，参考 LRU 顺序按参考语义维护，二者不混为一个状态。历史衰减、窗口长度、误差阈值和模型重标定规则由实现配置记录。

3.9 分配机制说明例

以下仅用假设参数说明算法计算步骤，不作为实验数据或性能结论。设可分页预算为 10 GiB，历史模块建议专家占 40%；设置 Prefill 倾向专家占 70%、Decode 倾向专家占 30%。当前批次有 3 个 Prefill 请求和 1 个 Decode 请求，则 Prefill 请求比例为 75%，当前建议为专家占 60%。若当前建议权重 λ 为 0.5，融合后的专家比例为 50%，初步容量为专家 5 GiB、KV 5 GiB，再根据对象大小取整并检查最低容量约束。

下一批次若变成 1 个 Prefill 和 3 个 Decode，在相同倾向配置下，当前建议变为专家占 40%。历史建议不必先等待新的执行时间或 miss 完全体现该变化，分配器已经可以利用排程元数据作出修正。该例说明信息流和计算方式，不证明请求比例足以准确描述全部工作集，也不证明这一调整在所有模型和负载上必然更快。

3.10 具体实施环境与实验结果

本实施例在单 GPU 与 CPU 主存组成的推理服务器上实现完整 KITE 执行链，使用 Qwen3-30B-A3B 的完整 BF16 权重、真实路由器和自回归生成。专家权重的主副本保存在 CPU 主存，固定模型部分位于 GPU；专家池和 KV 池在同一可分页预算内分配。硬件型号用于说明本实施例，本方法不限定于该型号或模型。

实施配置

表 1 服务器及运行配置

项目	实施配置
GPU 与主存	NVIDIA A100 PCIe 40GB 单卡；主存约 125 GiB
CPU	双路 Intel Xeon Gold 5318Y；合计 48 个物理核心、96 个逻辑线程；本实验固定使用 8 个 CPU 线程
软件及精度	Python 3.12.3；PyTorch 2.7.1+cu118；Transformers 4.51.3；BF16 模型权重与 GPU 计算；CPU 专家计算使用 FP32 副本
管理对象	单个专家权重约 9 MiB；KV 页包含 128 tokens 的全 48 层 K/V，单页 12 MiB；共享显存以 3 MiB 帧管理
可分页预算	分别设为 6 GiB 和 7 GiB；固定权重、激活及临时工作区另行记账，显存保留安全余量

项目	实施配置
历史及当前信号	每 16 批刷新历史曲线；当前建议权重为 0.65；Prefill 与 Decode 的专家份额锚点分别为 0.8 和 0.2
候选划分及副本缓存	专家份额候选为 0.1、0.25、0.5、0.75、0.9，投影到最近可行容量；CPU FP32 专家副本缓存上限为 8 GiB

可分页预算是人为设置的缓存压力条件，不表示 A100 的全部 40GB 显存均可用于两类对象，也不表示设备总显存自然耗尽。对象粒度、精度、线程数及执行器在两组之间一致。

每次完整工作负载在开始阶段执行四条路径的成本标定。专家路径使用 q 为 1、8、32 的样本，KV 路径使用上下文长度为 128、512 的样本；以实际完成时间建立常数或带截距的线性近似。没有 FP32 副本时计入冷转换开销。超出标定范围的情况保守采用专家上传或 KV 恢复，不静默外推。KV 备用重计算从原始及已生成 token 重放前缀，多块共享同一重放任务时只执行和计费一次。

请求负载与对照方法

构造四条同时到达的请求，输入长度依次为 8192、11264、11264、11264 tokens，每条生成 16 个真实输出 tokens，总输出为 64 tokens。提示正文为留档的手写文本，经重复和截断达到指定长度。使用真实路由与贪心自回归生成，统一忽略 EOS 提前终止以固定输出量，不人为指定专家或 routed-token 数。四条请求的位置和 KV 独立，注意力相互隔离；不使用 chunked prefill、答案缓存或前缀缓存。

本次负载共 16 个计算批次：首批包含四条 Prefill，后续 15 批包含四条 Decode。比较固定成本基线和完整 KITE 两种配置。固定成本基线使用同一启动样本的常数统计值，按历史曲线选择划分；KITE 使用常数或线性路径模型，并将历史建议与当前 P/D 请求比例修正融合。两组共用执行器、请求、预算、精度、fallback 集合和初始空 GPU 缓存。该基线是本实施例的固定成本历史分配对照，不作为原始 WiSP 实现。

在每个预算下，两种配置各执行一次完整工作负载。6 GiB 下先执行基线、后执行 KITE；7 GiB 下交换两组顺序。完成时间包含各组自己的启动标定、池初始化、全部 Prefill/Decode、在线统计和决策、容量调整、实际搬运、写回及过期释放。模型加载在工作负载开始前完成，

时间单独记录，不计入图 3 的工作负载完成时间。TPOT 按每条请求首个至末个输出的时间差除以 15 后，在同一负载的四条请求之间取均值。

实测结果

表 2 两种预算下的完整工作负载实测结果

指标	6 GiB 基线	6 GiB KITE	7 GiB 基线	7 GiB KITE
完整完成时间 s	82.174	75.348	78.091	75.634
其中主执行时间 s	51.646	48.006	50.382	47.978
TPOT s/token	1.535	1.403	1.541	1.392
实际 KV 恢复次数	4934	200	4934	179

如图 3 所示，6 GiB 预算下，完整工作负载耗时由 82.174 秒降至 75.348 秒，降低 8.31%，对应加速比约 1.091；7 GiB 下由 78.091 秒降至 75.634 秒，降低 3.15%。两组实际输出数量一致。6 GiB 的主执行时间由 51.646 秒降至 48.006 秒，表明收益不只发生在启动区间。

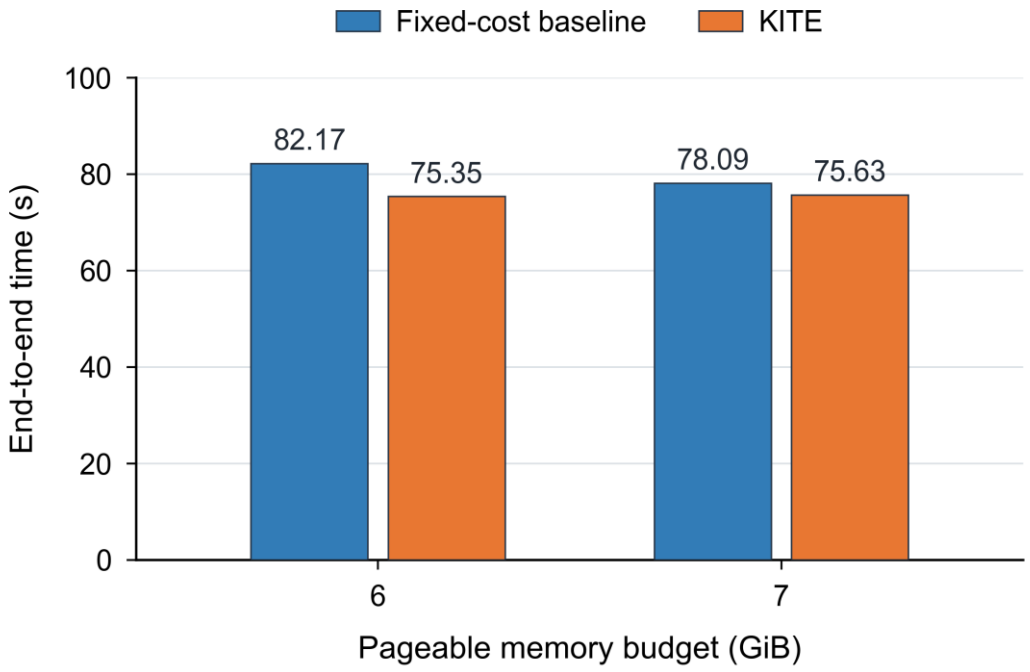


图 3 不同可分页预算下的完整工作负载完成时间

如图 4 所示，6 GiB 下 TPOT 由约 1.535 秒降至 1.403 秒，降低 8.59%；7 GiB 下由约 1.541 秒降至 1.392 秒，降低 9.68%。图中的数值为同一工作负载内四条请求的 TPOT 均值，不是多次独立试验的均值。

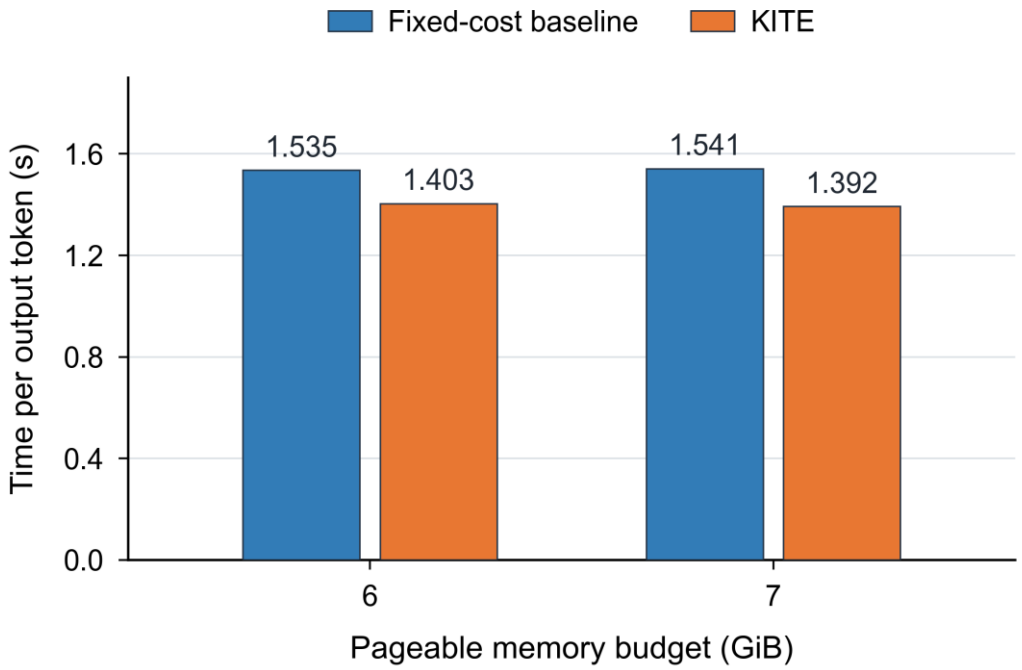


图 4 不同可分页预算下的输出 token 间隔

如图 5 所示，6 GiB 下实际 KV 恢复次数由 4934 次降至 200 次，减少 95.95%；7 GiB 下由 4934 次降至 179 次，减少 96.37%。所示次数来自实际执行的恢复事件，而不是复用距离曲线推算值。6 GiB 基线的 KV 容量始终为 256 页，KITE 在 Decode 阶段扩为 384 页；本次 KV 峰值为 332 页，因此 KITE 可以覆盖该工作集而基线不能。7 GiB 下对应容量为基线 298 页和 KITE Decode 448 页。

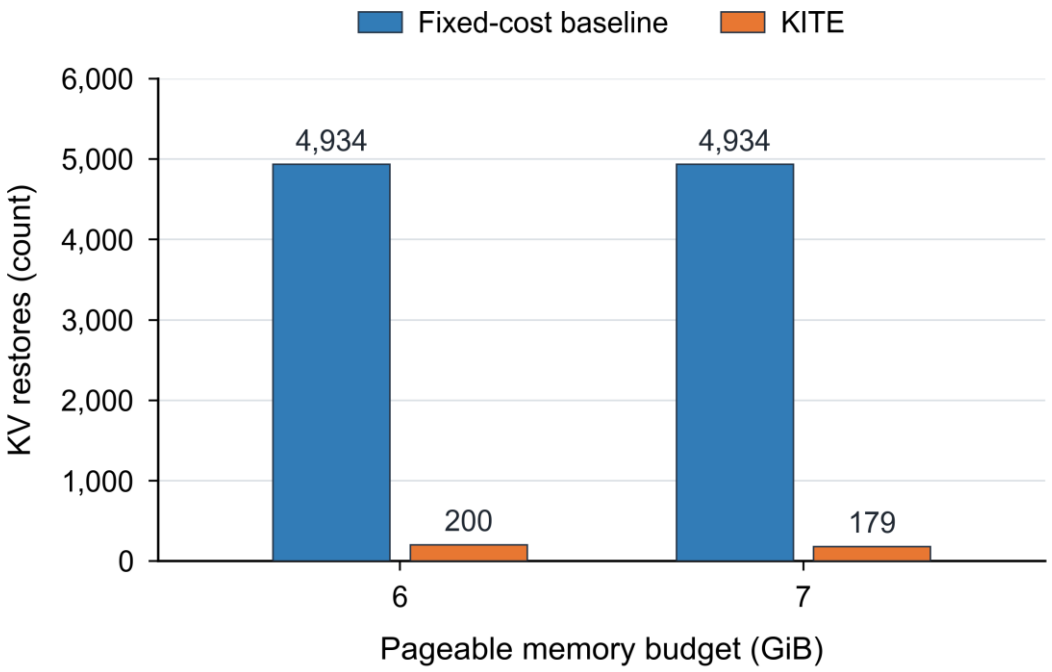


图 5 不同可分页预算下的实际 KV 恢复次数

本次两种配置的 Expert GPU 命中数均为零，缩小专家池没有损失已经存在的专家命中，从而避免了以增加专家恢复抵消 KV 收益的情况。日志记录的 6 GiB 实际 KV 恢复累计时间由约 2.884 秒降至 0.115 秒。以上实测支持完整策略在该长输入、短输出、四请求受控场景中的整体效果，不将全部完成时间差单独归因于某一模块，也不据此推断任意请求或长生成稳态下具有相同增益。

显存、主存及恢复事件均记录于运行日志。6 GiB 下两组 GPU 分配峰值均约 21.49 GiB，主机 KV 峰值为 3984 MiB，CPU FP32 副本峰值为 126 MiB，满足所设预算和安全余量；运行期间未观察到其他 GPU 计算进程干扰。

4、你的 idea 有什么技术优点？（与第 2 步中的技术缺点相对应）

针对历史统计响应滞后，本发明将已排程批次构成作为执行前可获得的信号，与历史建议联合使用，使分配器具备提前响应阶段组成变化的机制；不需要等该批次执行完毕后再观察时间占比。

针对统一 miss 成本难以反映路径差异，本发明分别维护四条路径的常数或参数化近似模型，将专家路由 token 数及 KV 恢复上下文等因素用于单次 miss 开销估计，使容量比较不只依赖 miss 总数。模型参数可通过实际完成时间持续维护。

针对阶段历史割裂与缓存语义混淆，本发明保留每类资源跨 P/D 的统一访问历史，同时区分参考 miss 曲线与实际缓存状态；在规定的 LRU 条件下可复用直方图的容量分析能力，在不满足条件时通过误差记录和保守控制限制过度依赖历史估计。

针对显存资源竞争和在线控制开销，本发明在同一字节预算内协调专家池与 KV 池，仅在候选划分之间进行低维比较与融合，并在安全点处理取整、驱逐和扩容。第 3.10 节实施例中，6 GiB 可分页预算下完整工作负载耗时降低 8.31%，TPOT 降低 8.59%，实际 KV 恢复次数减少 95.95%。这些结果体现了整体方案在所述受控场景下的技术效果，具体改善幅度随工作负载及系统配置变化。

5、本发明的关键技术点（请发明人列出其认为本发明最具创造性的技术点）

关键技术点一：在专家权重与 KV 缓存共用的显存预算下，把候选容量的 miss 估计与单次 miss 的路径代价估计解耦，再结合形成基于历史的联合分配建议。资源访问历史跨 Prefill/Decode 共享，而 cost 使用与路径计算和搬运相关的参数。

关键技术点二：在推理批次执行前读取排程元数据，以 Prefill/Decode 构成生成当前批次分配建议，并与历史建议加权融合，经可行容量投影后输出唯一的专家池与 KV 池划分。当前建议不依赖已被分配动作改变的 GPU 执行时间。

关键技术点三：将统一访问统计、路径计时、实际缓存状态和安全容量调整连接成在线反馈流程，使下一轮使用已完成事件形成的历史版本，同时明确参考 LRU 预测与真实准入语义之间的边界。

上述技术点应作为协同组合描述。复用距离统计、LRU 规则、CPU–GPU 卸载或某条 fallback 路径单独存在于已有技术中，本交底书不将其单独表述为本发明首次提出。拟保护的重点是其与当前批次阶段修正及联合显存分配的具体组合和信息流。

6、第 3 步中是否有可替代的方案（无，可以不填）

阶段信号的替代：以当前批次中的 query token 数或固定归一化工作量替代请求数，仍从执行前已知元数据生成修正建议。锚点插值可以替换为有限的阶段比例查表，表项由独立标定给出；不需要重新建立按阶段分开的物理缓存。

融合方式的替代：将加权后的目标比例映射到最近可行候选，或只在该比例邻近的候选集合内再次比较成本。误差过大时降低历史权重、暂时保留当前配置或启用安全固定划分。替代方式仍保留历史证据和当前批次信息共同参与决策。

cost 模型的替代：在常数或线性模型不能充分描述实测曲线时，采用分段线性或小规模查表，输入仍为对象大小、q、L 及固定执行配置。对于共享前缀的多个 KV 恢复对象，可采用对象组为粒度的宏恢复计费。

缓存估计的替代：在采用非 LRU、绕过准入或不同大小对象的系统中，可用符合实际准入规则的候选虚拟缓存估计替换参考直方图，或者固定为满足 LRU 前提的执行模式。该替代不改变四路径计时和历史与当前批次融合流程，但具体精度和维护开销需要单独验证。

7、专业术语解释（无，可以不填）

MoE（Mixture of Experts）：混合专家模型。每个 token 通过路由器选择部分专家执行前馈计算，未被选择的专家不参与该 token 的专家运算。

Expert 与 routed token：Expert 指一个专家子网络及其权重；routed token 指在一次专家调用中被路由到该专家的输入 token，数量记为 q。

KV Cache：注意力计算中保存历史 token 的 Key 和 Value 张量的缓存。上下文长度 L 按恢复协议定义为相关历史 token 数，而不是任意输入字符串长度。

Prefill 与 Decode：Prefill 处理新输入 token 或已排程输入分块；Decode 使用已形成的上下文计算后续生成 token。同一 GPU 批次可以包含两类请求。

miss 与 fallback：miss 指所需管理对象在对应 GPU 池中不可用；fallback 指满足该需求的补救路径，本方案包括专家 Upload、专家 CPU 计算后回传、KV Restore 和 KV Recompute。

复用距离与 miss 曲线：复用距离为两次访问同一对象之间访问的不同对象数；直方图统计各距离的出现次数，miss 曲线表示参考缓存容量与 miss 数或 miss 率的关系。

LRU 与准入：LRU 按最近访问顺序驱逐最久未使用对象；准入指 miss 处理后将对象纳入 GPU 缓存。CPU 执行但不上传专家权重属于绕过权重准入。

联合预算与分配比例：B 为扣除固定部分和预留后的可分页字节预算，B_E、B_K 为两池容量；f_h、f_c、f 分别表示历史、当前和融合后的专家预算比例；f_P、f_D 为阶段倾向锚点，λ 为当前建议权重。

Phase-aware 与 path-aware：前者指利用当前排程批次的阶段构成修正资源分配；后者指在代价估计中显式考虑不同恢复路径及其参数。KITE 为本联合分配方法的项目名称。

8、技术参考文献（无，可不填。包括之前申请的关联专利、论文，自己检索出来的近似技术专利等）

[1] Jiamu Zhang, Liang Wu, Mayank Darbari, Liangjie Hong. WiSP: A Working-Set View of Mixture-of-Experts Serving on Extremely Low-Resource Hardware. arXiv:2606.21868, 2026. https://arxiv.org/abs/2606.21868

[2] Keisuke Kamahori, Yile Gu, Kan Zhu, Baris Kasikci. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. arXiv:2402.07033, 2024. https://arxiv.org/abs/2402.07033

[3] Qwen Team. Qwen3-30B-A3B 模型说明及配置。https://huggingface.co/Qwen/Qwen3-30B-A3B